

WikiRMI

Rozproszony System Wiki — dokumentacja techniczna

Programowanie Współbieżne i Rozproszone (2026) · Temat 9 — system typu wiki

Tytuł	Autorzy
WikiRMI — Rozproszony System Wiki	Maciej Wawer, Krystian Wasił, Marek Więcaszek
Technologie	Repozytorium / gałąź
Java 17 (java.rmi, java.util.concurrent), Swing + FlatLaf	github.com/maciej-wawer/RMI · refaktor-wspolbieznosc

Podział pracy (zadaniowy): Maciej Wawer — architektura RMI i warstwa serwera; Krystian Wasił — rdzeń współbieżności (WikiStore: blokady, monitor, semafor; pula wątków powiadomień); Marek Więcaszek — klient Swing/FlatLaf, persystencja JSON, testy.

1. Cel i zakres

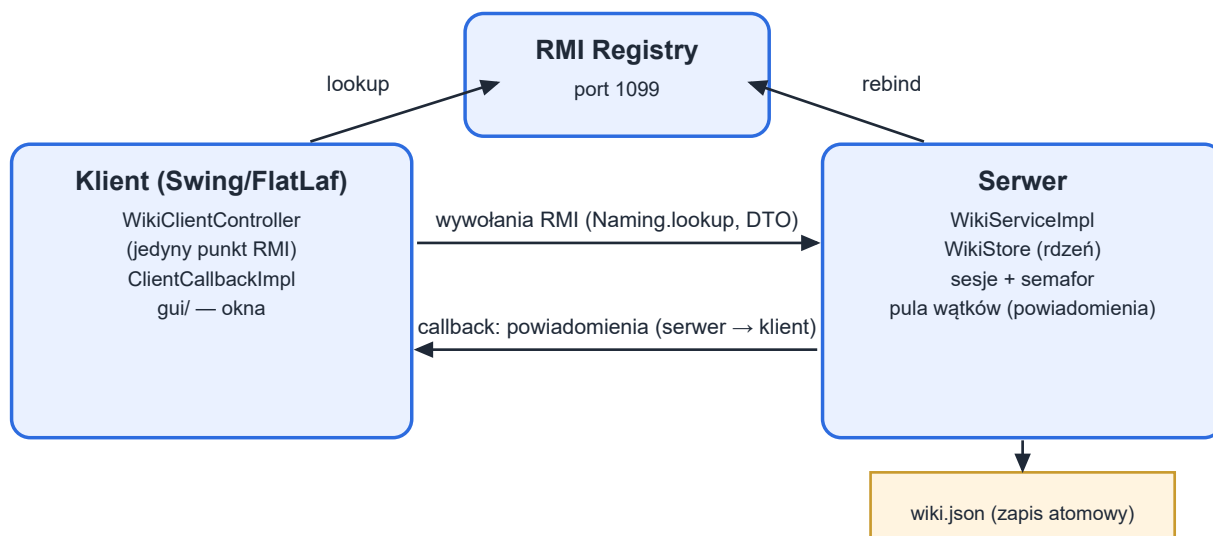
Aplikacja rozproszona w języku Java wykorzystująca **RMI** (zdalne wywoływanie metod) oraz mechanizmy **programowania równoległego**. Realizuje system typu wiki: administrator tworzy szkielety stron i konta, użytkownicy przeglądają, wyszukują i modyfikują strony. Rdzeniem projektu jest poprawna, drobnoziarnista i wolna od zakleszczeń współbieżność przy jednoczesnym dostępie wielu klientów.

Założenia

- Komunikacja klient–serwer wyłącznie przez RMI; dane przesyłane jako DTO implementujące `Serializable`.
- Rejestr RMI uruchamiany w procesie serwera (`LocateRegistry.createRegistry`), usługa rejestrowana przez `Naming.rebind`.
- Jedyna zewnętrzna biblioteka to FlatLaf (wygląd UI, `lib/flatlaf-3.5.4.jar`); reszta bez zależności — własny kodek JSON i własny zestaw testów (bez JUnit).
- Trwałość w czytelnym pliku JSON; po restarcie serwera stan jest w pełni odtwarzany.
- Ścisły rozdział warstw: kontrakt (`common`) — serwer (`server`) — klient (`client`).

2. Architektura

Budowa trójwarstwowa RMI. Serwer rejestruje obiekt zdalny w rejestrze (`Naming.rebind`), klient pobiera „namiasstkę” (stub) przez `Naming.lookup` i wywołuje metody jak lokalne. RMI działa też w drugą stronę — callbacki serwer→klient (powiadomienia na żywo).



Rys. 1. Schemat blokowy: Klient – RMI Registry – Serwer.

Warstwy

Warstwa / pakiet	Najważniejsze klasy i odpowiedzialność
common	Kontrakt: <code>WikiService</code> (extends <code>Remote</code>), <code>WikiClientCallback</code> ; DTO w jednym pliku <code>Dto</code> (klasy zagnieżdżone); wyjątki <code>WikiException</code> + <code>PageLockedException</code> .
server	<code>WikiServiceImpl</code> (extends <code>UnicastRemoteObject</code>); <code>WikiStore</code> — RDZEŃ WSPÓLBIEŻNOŚCI (blokady + monitor + semafor + sesje); <code>NotificationService</code> (pula wątków — powiadomienia); <code>JsonPersistence</code> ; <code>PasswordHasher</code> ; <code>WikiServer</code> .
client	<code>WikiClient</code> (FlatLaf); <code>WikiClientController</code> — jedyny punkt styku z RMI; <code>ClientCallbackImpl</code> ; <code>gui/</code> (Swing); <code>MarkdownRenderer</code> , <code>TextDiff</code> .

Konsolidacja: cały rdzeń współbieżności znajduje się w jednym pliku `WikiStore.java`, co ułatwia analizę. Sesje i limit klientów (semafor) oraz wątek sprząający blokady zostały wciągnięte do tej klasy.

3. Specyfikacja interfejsu zdalnego (WikiService)

Każda metoda zgłasza RemoteException (błąd sieci/RMI) lub WikiException (błąd domenowy). Wszystkie poza login przyjmują token sesji.

Metoda	Opis	Uprawnienia
login / logout	Uwierzytelnienie i zakończenie sesji.	wszyscy
createUser / deleteUser / listUsers	Zarządzanie kontami; usunięcie wylogowuje i zwalnia blokady (kickUser).	admin
createPage	Utworzenie strony.	zalogowani
deletePage / forceUnlock	Usunięcie strony / wymuszone zdjęcie blokady.	admin
listPages / searchPages / getPage / listOnlineUsers	Lista, wyszukiwanie, odczyt strony, lista online.	wszyscy
acquireEditLock / renewEditLock / releaseEditLock / savePage	Pozyskanie, odnowienie, zwolnienie blokady edycji oraz zapis.	wszyscy / posiadacz
getHistory / getRevision / restoreRevision	Historia, podgląd i przywracanie rewizji.	wszyscy
changePassword	Zmiana własnego hasła.	wszyscy
subscribe / unsubscribe	Rejestracja powiadomień (callback).	wszyscy

Callbacki (WikiClientCallback): onPageCreated, onPageChanged, onPageDeleted, onLockChanged, onPresenceChanged — odświeżają GUI innych klientów w czasie rzeczywistym.

4. Model współbieżności i synchronizacji

Serwer RMI obsługuje każde wywołanie klienta w osobnym wątku, więc te same dane (np. jedna strona) są dotykane przez wiele wątków. Sekcja krytyczna to jednoczesna edycja tej samej strony — bez ochrony wystąpiłby stan wyścigu lub utrata zapisu. Zastosowano wszystkie cztery mechanizmy — trzy w pliku WikiStore (blokady, monitor, semafor), a czwarty (pula wątków) w NotificationService:

Mechanizm	Plik : linia	Rola
BLOKADY — ReentrantReadWriteLock (per strona)	Page.java:23; WikiStore.java: writeLock 350/407, readLock 301	Wielu czytelników, wyłączny zapis; sekcja krytyczna „sprawdź-i-ustaw”.
MONITORY — synchronized	WikiStore.java:133,140	Wspólny licznik zapisów; niepodzielna inkrementacja.
SEMAFORY — Semaphore	WikiStore.java:51, 86, 113	Limit jednoczesnych klientów (sesje).
WĄTKI — pula wątków (ExecutorService)	NotificationService.java:21,29	Powiadomienia (callbacki RMI) wysyłane poza wątkiem edycji; wolny/martwy klient nie blokuje zapisu.
ConcurrentHashMap + putIfAbsent	WikiStore.java:43–45, 212, 272	Atomowe mapy i tworzenie stron/użytkowników.
volatile	WikiStore.java:78	Bezpieczna widoczność pola sesji między wątkami.

Sekcja krytyczna — „sprawdź i ustaw” blokadę edycji

```
public LockInfoDTO acquireEditLock(String title, String token, String userName) { // WikiStore.java
    Page p = require(title); long now = clock.now();
    p.lock().writeLock().lock(); // BLOKADA zapisu – wpuszcza jeden wątek
    try {
        EditLock cur = p.editLock();
        if (cur != null && !cur.isExpired(now) && !cur.heldBy(token))
            throw new PageLockedException(...); // przegrani z N wątków
        p.setEditLock(new EditLock(token, userName, now, now + leaseMs)); // zwycięzca
        return toLockInfo(...);
    } finally { p.lock().writeLock().unlock(); }
}
```

Brak zakleszczeń, brak utraconych aktualizacji

- Brak deadlocka: każda operacja trzyma najwyżej JEDNĄ blokadę strony naraz i nie zagnieżdża blokad — nie powstaje cykl oczekiwań.
- Brak lost update: savePage działa pod wyłącznym writeLock i dodatkowo sprawdza numer wersji (kontrola optymistyczna); inkrementacja wersji i dopis do historii są niepodzielne.
- RMI Naming: serwer LocateRegistry.createRegistry + Naming.rebind("//host:port/WikiService") (WikiServer.java:35–36); klient Naming.lookup (WikiClientController.java:35).

5. Zarządzanie pamięcią, wydajność i odporność

- Przeterminowane blokady edycji są wykrywane leniwie przy acquireEditLock / renewEditLock / savePage (isExpired) — zawieszony lub zamknięty klient nigdy nie blokuje strony na stałe, bez potrzeby osobnego wątku sprzątającego.
- Powiadomienia (callbacki RMI) wysyłane na puli wątków (ExecutorService) — wolny/„martwy” klient nie blokuje wątku edycji; klient zgłaszający RemoteException jest usuwany.
- Persystencja write-through: po każdej zmianie zapis do pliku tymczasowego i atomowa podmiana (ATOMIC_MOVE) + hak zamknięcia.
- Usunięcie konta (kickUser) zamyka sesje (oddaje pozwolenie semafora) i zwalnia blokady usuwanego użytkownika.

- Nieważna sesja po stronie klienta (np. konto usunięte, restart serwera) powoduje automatyczne wylogowanie do ekranu logowania (`WikiException.SESSION_INVALID` + `UiUtils` → `MainFrame.forcedLogout`).
- Klient wykonuje wywołania RMI poza wątkiem EDT (`SwingWorker`) — interfejs nie zamarza.

6. Funkcjonalność

- Logowanie z rolami (administrator / użytkownik), limit jednoczesnych klientów (semafor).
- Przeglądanie i wyszukiwanie pełnotekstowe stron.
- Edycja z blokadą (jeden edytor naraz), licznik czasu blokady, podgląd Markdown i klikalne linki `[[Strona]]`.
- Powiadomienia na żywo (callbacki RMI): panel online + aktualnie edytowane, automatyczne odświeżanie.
- Historia wersji: przeglądanie rewizji, porównanie różnic (diff) i przywracanie.
- Administracja: tworzenie/usuwanie kont i stron, wymuszone odblokowanie; zmiana hasła; wygląd FlatLaf.

7. Instrukcja uruchomienia (krok po kroku)

Wymagania: JDK 17 (testowano na Amazon Corretto 17). Skrypty wyszukują JDK w `%USERPROFILE%\jdk`s lub w `JAVA_HOME` i dołączają `lib/*.jar` (FlatLaf) do classpath.

```
.\build.ps1          # kompilacja src + test do out/ (UTF-8, FlatLaf na classpath)
.\run-server.ps1     # serwer: rejestr RMI na porcie 1099, Naming.rebind, wczytanie/seed danych
.\run-client.ps1     # klient (Swing/FlatLaf) – uruchom kilka instancji dla demonstracji
.\run-tests.ps1      # 5 testów współbieżności
```

Domyślne konto administratora przy pierwszym uruchomieniu: **admin / admin123**.

8. Testy współbieżności i wyniki

Pięć testów stricte współbieżnościowych (każdy = jeden scenariusz, z polską narracją kroków). Wynik zbiorczy: **5 passed, 0 failed**.

Test	Co sprawdza	Wynik
Demo1 – usuwanie edytowanej strony	Konflikt usunięcia z edycją; spójność stanu.	PASS — zapis odrzucony bez awarii
Demo2 – wymuszone odblokowanie	Admin zdejmuję cudzą blokadę.	PASS — inny wchodzi
Demo3 – dwóch naraz nie wejdzie	Wyścig: 2 osoby + 10 wątków naraz.	PASS — weszło 1, odrzuconych 9
Demo4 – usunięcie edytującego	Usunięcie konta: wylogowanie + zwolnienie blokad (semafor + blokada).	PASS
Demo5 – brak utraconych zapisów	50 wątków zapisuje tę samą stronę.	PASS — wersja = 50

Ograniczenia: pojedynczy serwer (jeden węzeł); kodek JSON obsługuje wymagany podzbiór typów; render Markdown obejmuje wybrany podzbiór składni; maksymalna liczba klientów ograniczona semaforem (domyślnie 50, konfigurowalne w `ServerConfig`). Po restarcie serwera stan jest w pełni odtwarzany z pliku `wiki.json`; blokady edycji są ulotne i nie są utrwalane.

9. Podsumowanie

WikiRMI to kompletny system rozproszony RMI z poprawną, drobnoziarnistą i wolną od zakleszczeń współbieżnością skupioną w jednym pliku (`WikiStore`) i pokazaną na czterech mechanizmach. Całość pokryta pięcioma testami współbieżności (w tym wyścigiem 10 wątków i testem braku utraconych

zapisów). Kod: github.com/maciej-wawer/RMI (gałąź refaktor-wspolbieznosc).